TP 3 - Architecture distribuée

Encadré par: Pr. BE ELBAGHAZAOUI

Réalisé par : Nabila Abouilaaz

Introduction

Ce TP a pour objectif de mettre en place une architecture distribuée complète pour la gestion de commandes. Contrairement à une application monolithique simple, une architecture distribuée repose sur plusieurs composants spécialisés qui coopèrent ensemble. Dans ce travail, nous avons utilisé plusieurs services complémentaires : deux instances d'une application Node.js/Express, un load balancer Nginx, une base de données MongoDB, un cache Redis ainsi qu'un système de messagerie RabbitMQ.

Le but principal est de comprendre le rôle de chaque composant dans une architecture moderne. L'application permet de créer et consulter des commandes, de répartir la charge entre plusieurs instances du serveur, d'accélérer les lectures grâce au cache Redis et de traiter les notifications de manière asynchrone avec RabbitMQ.

Ce TP illustre donc de manière pratique plusieurs notions fondamentales des systèmes distribués : répartition de charge, tolérance aux pannes, séparation des responsabilités, mise en cache et communication asynchrone.

Objectifs du TP

Les objectifs de ce TP sont les suivants :

- déployer une architecture distribuée à l'aide de Docker Compose ;
- mettre en place deux instances d'une application Node.js/Express ;
- configurer Nginx comme load balancer ;
- utiliser MongoDB pour la persistance des commandes ;
- utiliser Redis comme cache pour accélérer les lectures ;
- utiliser RabbitMQ pour la communication asynchrone ;
- observer la différence entre une lecture depuis MongoDB et une lecture depuis Redis ;
- comprendre le rôle de chaque composant dans l'architecture.

Architecture générale réalisée

L’architecture mise en place est composée des éléments suivants :

- **Client / Postman** : permet d’envoyer les requêtes HTTP à l’application ;
- **Nginx** : reçoit les requêtes du client et les distribue entre les deux instances applicatives ;
- **App1 et App2** : deux instances Node.js/Express qui traitent les requêtes ;
- **MongoDB** : base de données NoSQL utilisée pour stocker les commandes ;
- **Redis** : système de cache utilisé pour accélérer les lectures répétées ;
- **RabbitMQ** : broker de messages utilisé pour l’envoi d’événements de commande ;
- **Consumer** : service consommateur qui lit les messages RabbitMQ et simule l’envoi d’une notification.

Le flux général est le suivant :

1. le client envoie une requête à Nginx ;
2. Nginx la transmet à l’une des deux instances Node.js ;
3. lors d’une création de commande, l’instance enregistre la commande dans MongoDB ;
4. l’application publie ensuite un message dans RabbitMQ ;
5. le service consumer consomme ce message et affiche une notification simulée ;
6. lors de la consultation des commandes, l’application vérifie d’abord Redis ;
7. si les données ne sont pas en cache, elles sont lues depuis MongoDB puis mises en cache dans Redis.

Cette organisation permet d’obtenir une architecture plus performante, plus souple et plus proche des architectures distribuées modernes.

Technologies utilisées

Pour réaliser ce TP, les technologies suivantes ont été utilisées :

- **Docker Compose** pour orchestrer tous les services ;
- **Node.js / Express** pour développer l'API ;
- **MongoDB** pour le stockage des commandes ;
- **Redis** pour la mise en cache ;
- **RabbitMQ** pour la communication asynchrone ;
- **Nginx** comme load balancer ;
- **Postman** pour tester les endpoints ;
- **Docker Desktop** pour exécuter les conteneurs.

Structure du projet

Le projet a été organisé selon une structure simple et claire. On retrouve :

- un fichier `docker-compose.yml` pour définir tous les services ;
- un dossier `nginx` contenant le fichier `nginx.conf` ;
- un dossier `app` contenant le code Node.js ;
- dans `app`, des sous-dossiers `routes` et `services` ;
- un fichier `consumer.js` pour le consommateur RabbitMQ.

Cette organisation permet de séparer proprement la configuration, le serveur principal, les routes et les services techniques

Mise en place de Docker Compose

Le fichier docker-compose.yml a permis de lancer automatiquement tous les composants de l'architecture :

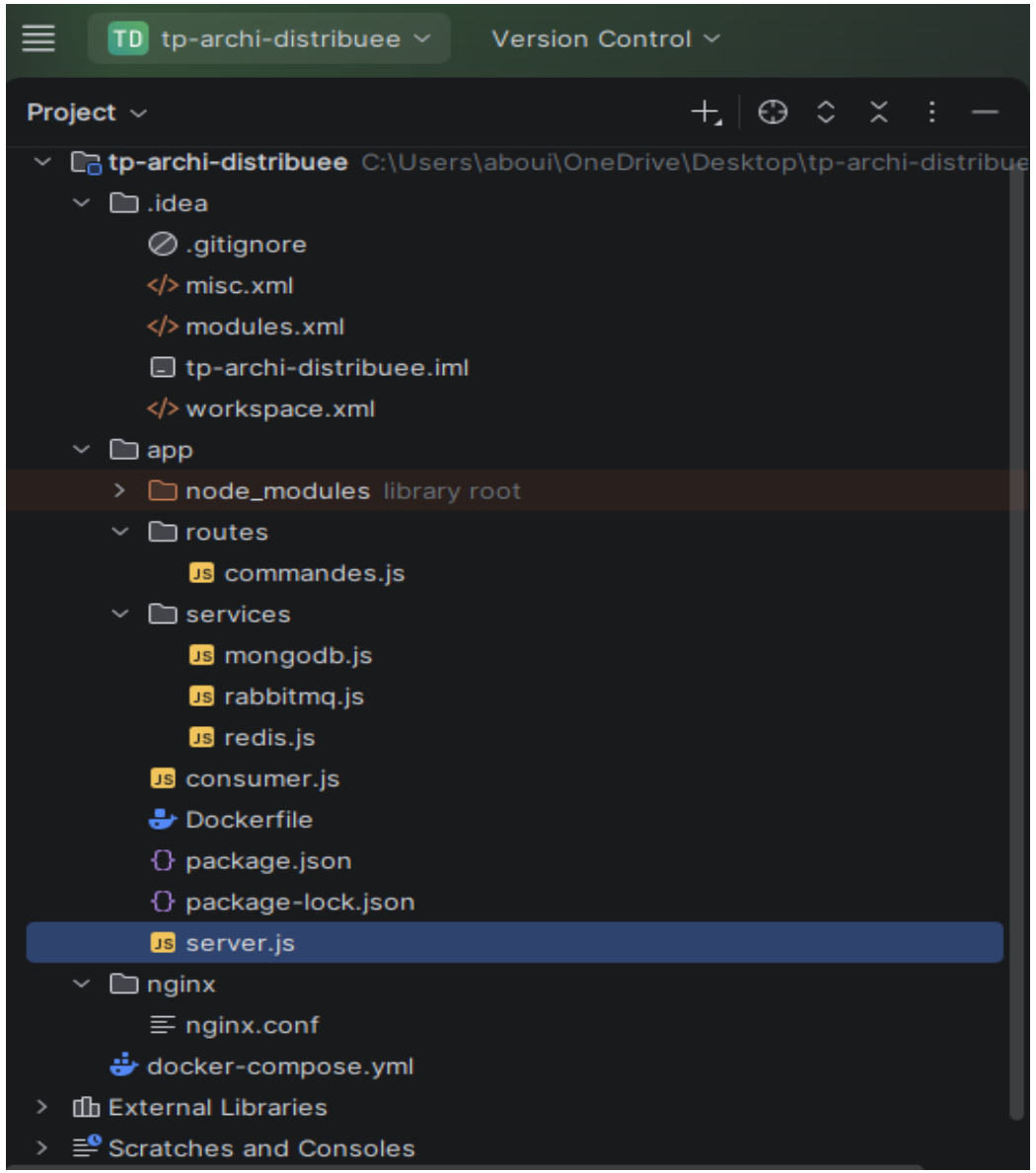
- MongoDB
- Redis
- RabbitMQ
- App1
- App2
- Nginx
- Consumer

Chaque service a été configuré avec son image Docker, ses ports et ses dépendances. Grâce à cette approche, l'ensemble du système peut être démarré avec une seule commande.

La commande utilisée est :

docker compose up –build

Cette étape a permis de construire les images nécessaires puis de lancer tous les conteneurs.



Configuration de Nginx

Nginx a été configuré pour écouter sur le port 80 et distribuer les requêtes vers les deux instances applicatives app1 et app2.

Le rôle de Nginx dans ce TP est très important, car il agit comme **point d'entrée unique** pour le client. Le client n'a pas besoin de savoir quelle instance traite la requête. Il envoie simplement ses requêtes à localhost, et Nginx les répartit automatiquement.

Après configuration, les endpoints suivants ont été testés avec succès :

- http://localhost:3001/health
- http://localhost:3002/health
- http://localhost/health

Les deux premières URLs permettent de tester chaque instance directement, tandis que la troisième prouve que la requête passe bien par Nginx.

Mise en place des instances Node.js / Express

Deux instances de l'application ont été déployées :

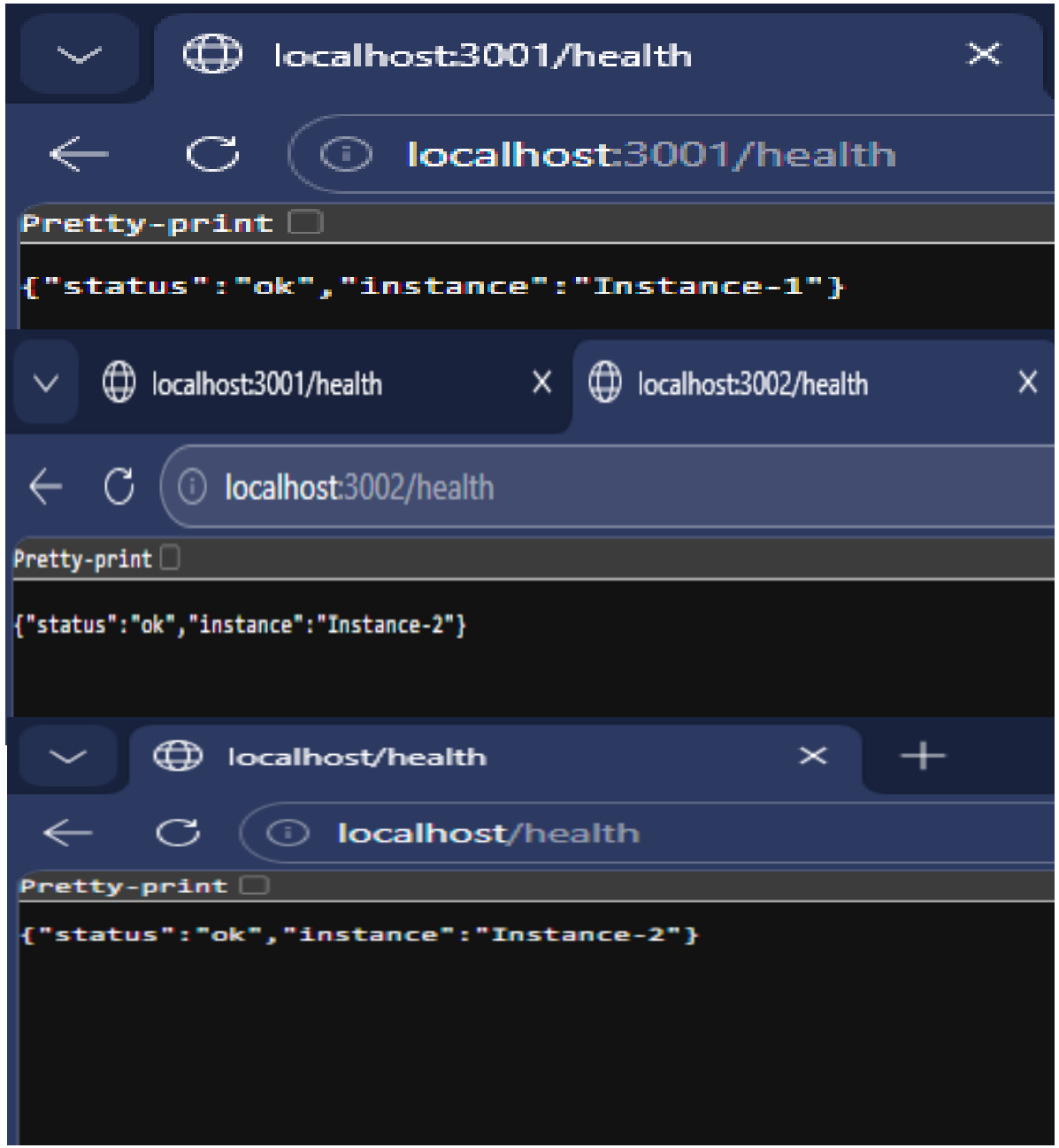
- **Instance-1**
- **Instance-2**

Chaque instance possède les mêmes routes et les mêmes fonctionnalités, mais elles sont distinguées par une variable d'environnement qui permet d'indiquer dans les réponses quelle instance a traité la requête.

Une route /health a été ajoutée pour vérifier le bon fonctionnement des serveurs. Les résultats obtenus montrent que :

- `localhost:3001/health` retourne `Instance-1`
- `localhost:3002/health` retourne `Instance-2`
- `localhost/health` retourne alternativement `Instance-1` ou `Instance-2`

Cela confirme que les deux serveurs fonctionnent correctement et que la répartition de charge est bien active.

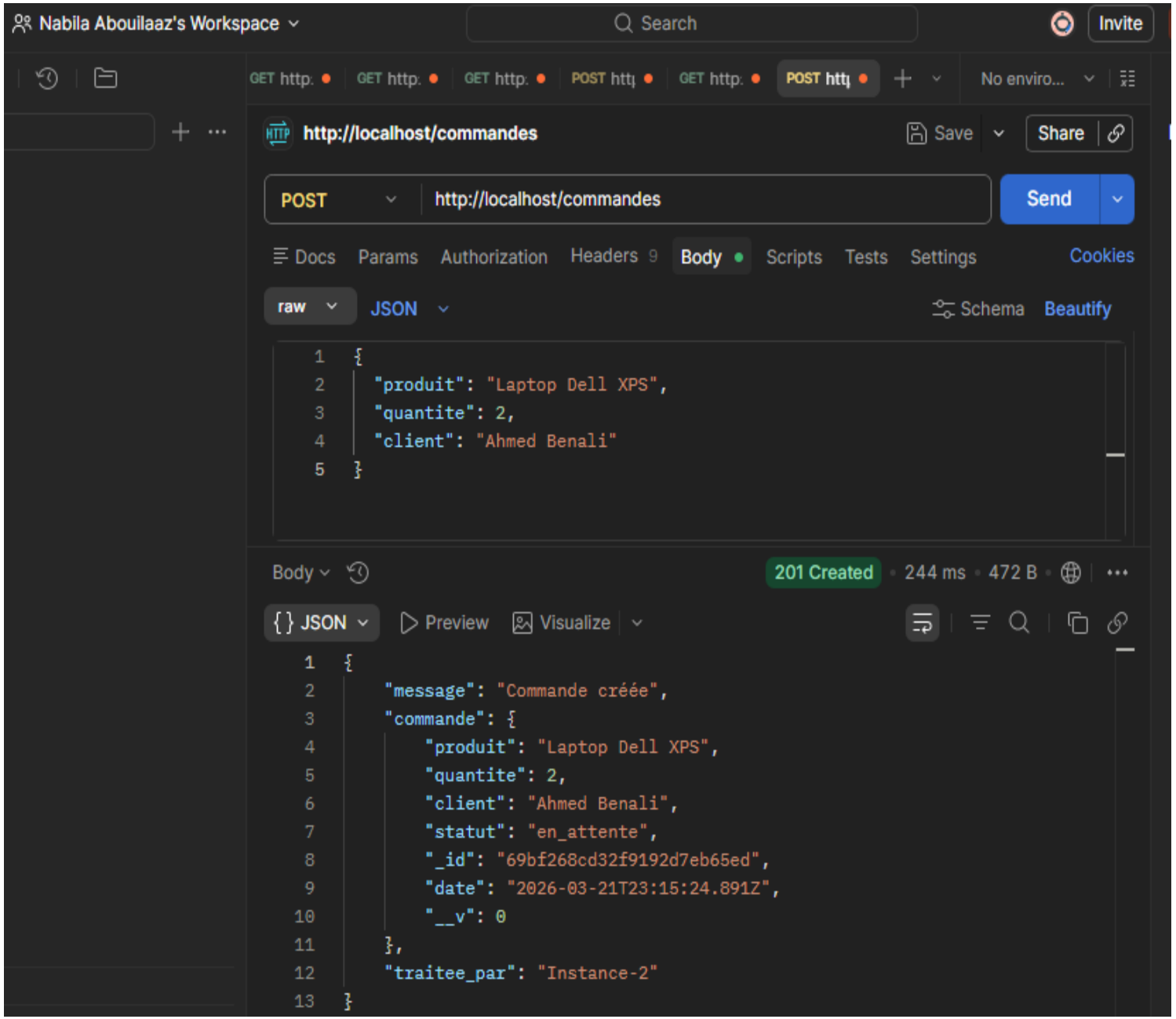


Intégration de MongoDB

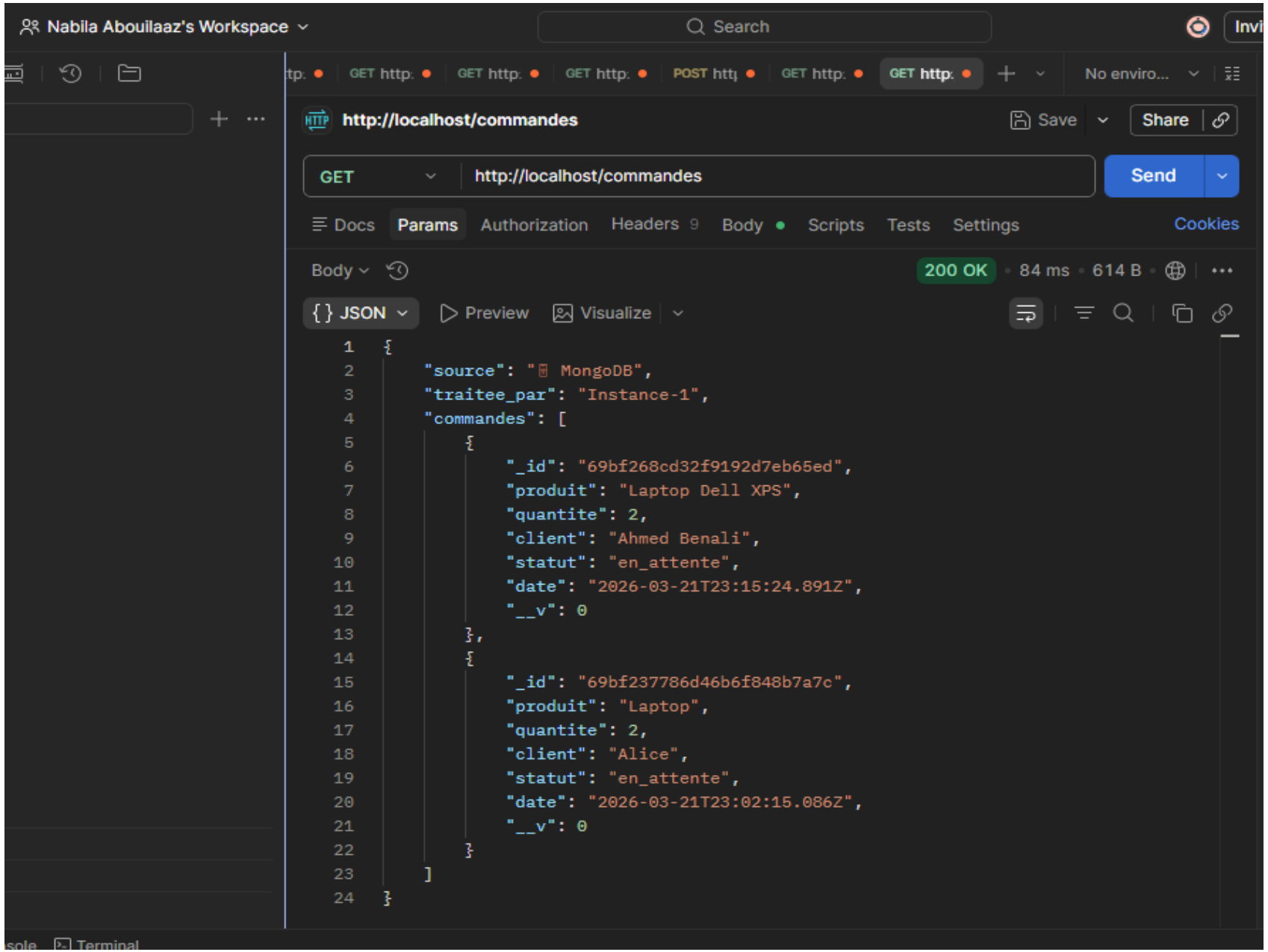
MongoDB a été utilisé comme base de données principale pour stocker les commandes. Chaque commande contient les champs suivants :

- produit
- quantite
- client
- statut
- date

Lorsqu'une requête POST /commandes est envoyée, la commande est enregistrée dans MongoDB. Cette étape garantit la persistance des données.



- Les tests ont montré que les commandes créées sont bien conservées et peuvent être récupérées ensuite via l'endpoint GET /commandes.



Intégration de Redis

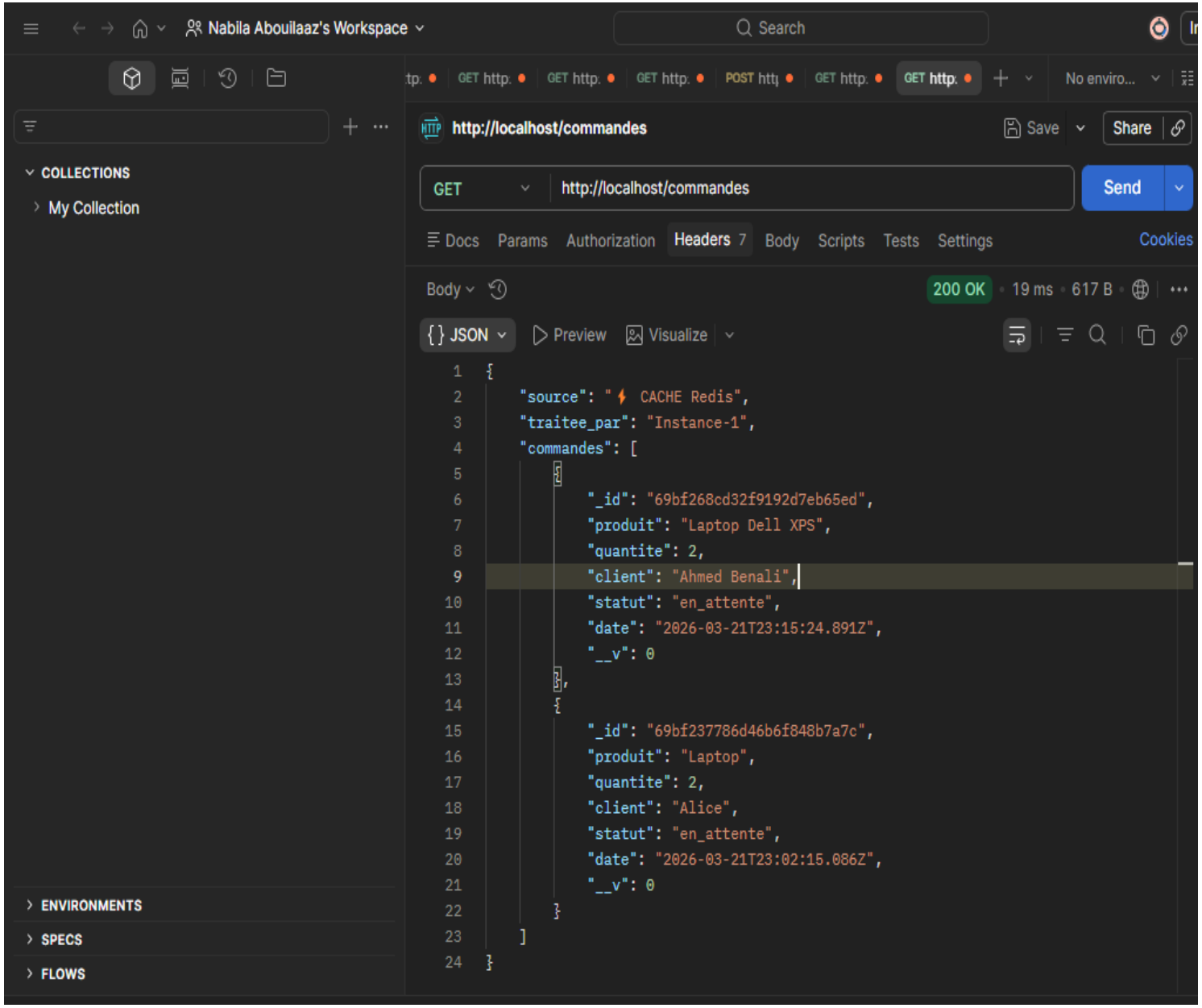
Redis a été utilisé comme système de cache pour les lectures des commandes.

Le fonctionnement attendu est le suivant :

- au **premier appel** de GET /commandes, les données proviennent de MongoDB ;
- ensuite, ces données sont stockées dans Redis ;
- au **deuxième appel**, les données doivent être servies depuis Redis.

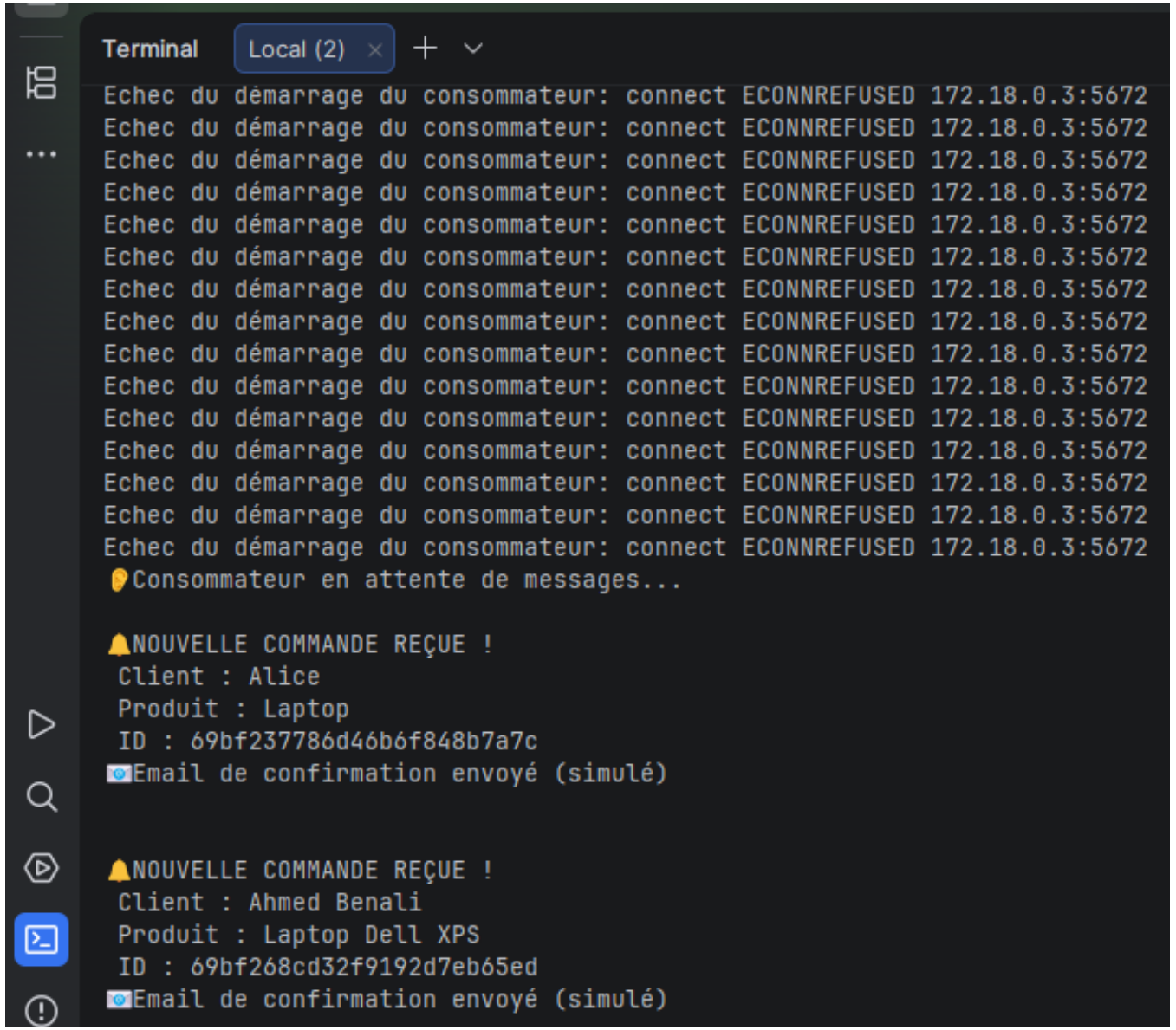
L'intérêt de Redis est d'améliorer les performances en évitant des requêtes répétitives à la base de données. Cette technique est très utilisée dans les architectures distribuées pour réduire la charge sur la base de données et accélérer les réponses.

Dans notre implémentation, le premier appel a bien montré la source MongoDB.



Intégration de RabbitMQ

RabbitMQ a été utilisé pour la communication asynchrone. Lors de la création d'une commande, l'application publie un message contenant les informations essentielles de la commande.
Ce message est ensuite lu par un service consumer, indépendant de l'application principale.
Le consommateur simule l'envoi d'un email de confirmation.
Cette partie est particulièrement importante, car elle montre que certaines tâches peuvent être traitées
de manière asynchrone, sans bloquer la requête principale. Ainsi, l'utilisateur reçoit rapidement une réponse à sa création de commande, tandis que la notification est traitée en arrière-plan par le consommateur.



Analyse des composants

- **Rôle de Nginx**

Nginx sert de load balancer. Il reçoit d'abord les requêtes du client, puis les distribue vers app1 ou app2.

Cela permet de mieux répartir la charge et d'améliorer la disponibilité de l'application.

- **Rôle de MongoDB**

MongoDB stocke les commandes de manière persistante. C'est la source principale de vérité pour les données de commande.

- **Rôle de Redis**

Redis stocke temporairement les résultats des lectures afin d'éviter des accès répétés à MongoDB. Il sert donc à accélérer les réponses et à améliorer les performances.

- **Rôle de RabbitMQ**

RabbitMQ sert de broker de messages. Il permet de découpler la création de commande et l'envoi des notifications.

- **Rôle du consumer**

Le consumer lit les messages déposés dans RabbitMQ et simule l'envoi d'une confirmation. Il représente ici

un microservice de notification séparé.

Réponses aux questions d’analyse

1. Quel composant reçoit en premier les requêtes du client ? Quel algorithme utilise-t-il ?

Le premier composant qui reçoit les requêtes du client est **Nginx**. Il agit comme load balancer. L’algorithme utilisé dans cette configuration est le **round-robin**, ce qui signifie que les requêtes sont réparties alternativement entre les deux instances disponibles.

2. Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ? Pourquoi ?

Oui, Nginx peut continuer à fonctionner tant qu’au moins une instance applicative reste disponible. Si app1 tombe en panne, Nginx peut continuer à transmettre les requêtes vers app2. Cela améliore la tolérance aux pannes et la disponibilité du système.

3. Expliquez la différence entre le premier GET (source MongoDB) et le deuxième GET (source Redis).

Lors du premier GET, les données ne sont pas encore présentes dans le cache. L’application doit donc interroger MongoDB. Une fois les données récupérées, elles sont enregistrées dans Redis. Lors du deuxième GET, l’application retrouve les données directement dans Redis, ce qui permet une réponse plus rapide.

4. Pourquoi invalide-t-on le cache Redis après un POST /commandes ?

On invalide le cache après un POST car la liste des commandes a changé. Si le cache n’était pas supprimé ou mis à jour, les prochaines lectures risqueraient d’afficher des données obsolètes. L’invalidation du cache garantit la cohérence des données.

5. En quoi RabbitMQ améliore-t-il la résilience du système ?

RabbitMQ améliore la résilience car il sépare le traitement principal de la commande et les tâches secondaires comme la notification. Même si la notification prend du temps ou échoue temporairement, la commande peut être créée et enregistrée normalement. Le système devient donc plus souple et plus robuste.

6. Si on voulait ajouter une troisième instance Node.js, quelle partie faudrait-il modifier ?

Il faudrait ajouter un nouveau service dans docker-compose.yml, par exemple app3, puis ajouter ce serveur dans le bloc upstream backend du fichier nginx.conf. Ainsi, Nginx pourrait répartir les requêtes sur trois instances au lieu de deux.

Difficultés rencontrées

Au cours de la réalisation de ce TP, plusieurs difficultés techniques ont été rencontrées :

- démarrage incomplet de Docker Desktop au début ;
- conflits d'images Docker lors du rebuild ;
- difficulté de communication entre Nginx et les instances applicatives ;
- erreur `502 Bad Gateway` avant correction de la configuration Nginx ;
- nécessité de forcer l'application Node.js à écouter sur `0.0.0.0` dans Docker ;
- erreurs de connexion temporaires du consumer à RabbitMQ au démarrage.

Ces problèmes ont été progressivement corrigés en ajustant la configuration Docker Compose, le fichier `nginx.conf` et le code du serveur Node.js.

Conclusion

En conclusion, ce TP a permis de déployer et tester une véritable architecture distribuée de gestion de commandes. L'utilisation combinée de Nginx, Node.js, MongoDB, Redis et RabbitMQ montre clairement comment plusieurs services peuvent collaborer dans un système moderne.

Ce travail a permis de comprendre concrètement :

- la répartition de charge avec Nginx ;
- l'intérêt de disposer de plusieurs instances d'application ;
- le rôle de MongoDB comme base de persistance ;
- l'importance du cache Redis pour les performances ;
- l'intérêt de RabbitMQ pour le traitement asynchrone.

Ainsi, ce TP constitue une excellente introduction pratique aux architectures distribuées et aux systèmes basés sur des services spécialisés.